

DAA Lab - Experiment 3

Implement Strassen's Matrix Multiplication algorithm and compare its performance with Conventional Matrix Multiplication

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement matrix multiplication using:
 - 1 Conventional method ($O(n^3)$)
 - 2 Strassen's method ($O(n^{\log_2 7}) \approx O(n^{2.81})$)
- Compare execution times for matrices of increasing order.

- Conventional multiplication:

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

- Strassen's Algorithm:
 - Divide A and B into four submatrices each.
 - Compute 7 multiplications ($M1$ to $M7$) instead of 8.
 - Combine results into the final matrix C .
- Reduces complexity from $O(n^3)$ to $O(n^{2.81})$.

Algorithm: Conventional Multiplication

Conventional_Multiply(A, B, n)

- ① For $i = 0$ to $n-1$:
- ② For $j = 0$ to $n-1$:
- ③ $C[i][j] = 0$
- ④ For $k = 0$ to $n-1$:
- ⑤ $C[i][j] += A[i][k] * B[k][j]$

Algorithm: Strassen's Multiplication (For REFERENCE)

Strassen_Multiply(A, B, n)

- 1 If $n = 1$: return $A[0][0] * B[0][0]$
- 2 Divide A and B into 4 submatrices of size $n/2$
- 3 Compute:
 - $M1 = (A11 + A22)(B11 + B22)$
 - $M2 = (A21 + A22)B11$
 - $M3 = A11(B12 - B22)$
 - $M4 = A22(B21 - B11)$
 - $M5 = (A11 + A12)B22$
 - $M6 = (A21 - A11)(B11 + B12)$
 - $M7 = (A12 - A22)(B21 + B22)$
- 4 Compute submatrices of C:
 - $C11 = M1 + M4 - M5 + M7$
 - $C12 = M3 + M5$
 - $C21 = M2 + M4$
 - $C22 = M1 - M2 + M3 + M6$

C Code - Conventional Multiplication (For REFERENCE)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void conventionalMultiply(int n, int A[][n], int B[][n], int C[][n]) {
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            C[i][j] = 0;
            for (int k=0; k<n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

C Code - Strassen's Multiplication (Recursive) (For REFERENCE)

```
void add(int n, int A[][n], int B[][n], int C[][n]) {
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j] = A[i][j] + B[i][j];
}

void subtract(int n, int A[][n], int B[][n], int C[][n]) {
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j] = A[i][j] - B[i][j];
}

void strassen(int n, int A[][n], int B[][n], int C[][n]) {
    if (n == 1) {
        C[0][0] = A[0][0] * B[0][0];
        return;
    }
}
```

```
int k = n/2;
int A11[k][k], A12[k][k], A21[k][k], A22[k][k];
int B11[k][k], B12[k][k], B21[k][k], B22[k][k];
int C11[k][k], C12[k][k], C21[k][k], C22[k][k];
int M1[k][k], M2[k][k], M3[k][k], M4[k][k];
int M5[k][k], M6[k][k], M7[k][k];
int T1[k][k], T2[k][k];
```

```
// Split matrices
for(int i=0;i<k;i++)
    for(int j=0;j<k;j++) {
        A11[i][j] = A[i][j];
        A12[i][j] = A[i][j+k];
        A21[i][j] = A[i+k][j];
        A22[i][j] = A[i+k][j+k];
        B11[i][j] = B[i][j];
        B12[i][j] = B[i][j+k];
        B21[i][j] = B[i+k][j];
        B22[i][j] = B[i+k][j+k];
    }
```

```
// M1 = (A11+A22)(B11+B22)
add(k, A11, A22, T1);
add(k, B11, B22, T2);
strassen(k, T1, T2, M1);

// M2 = (A21+A22)B11
add(k, A21, A22, T1);
strassen(k, T1, B11, M2);

// M3 = A11(B12-B22)
subtract(k, B12, B22, T1);
strassen(k, A11, T1, M3);

// M4 = A22(B21-B11)
subtract(k, B21, B11, T1);
strassen(k, A22, T1, M4);

// M5 = (A11+A12)B22
add(k, A11, A12, T1);
strassen(k, T1, B22, M5);
```

```

// M6 = (A21-A11)(B11+B12)
subtract(k, A21, A11, T1);
add(k, B11, B12, T2);
strassen(k, T1, T2, M6);
// M7 = (A12-A22)(B21+B22)
subtract(k, A12, A22, T1);
add(k, B21, B22, T2);
strassen(k, T1, T2, M7);
// Compute submatrices of C
for(int i=0;i<k;i++)
    for(int j=0;j<k;j++) {
        C11[i][j] = M1[i][j] + M4[i][j] - M5[i][j] + M7[i][j];
        C12[i][j] = M3[i][j] + M5[i][j];
        C21[i][j] = M2[i][j] + M4[i][j];
        C22[i][j] = M1[i][j] - M2[i][j] + M3[i][j] + M6[i][j];
    }
for(int i=0;i<k;i++)    // Combine into C
    for(int j=0;j<k;j++) {
        C[i][j] = C11[i][j];
        C[i][j+k] = C12[i][j];
        C[i+k][j] = C21[i][j];
        C[i+k][j+k] = C22[i][j];
    }
}

```

Sample Input/Output

Input (n=2):

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Output:

Conventional: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Strassen: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Complexity Analysis

- Conventional Multiplication:
 - Time: $O(n^3)$
 - Space: $O(1)$
- Strassen's Multiplication:
 - Time: $O(n^{\log_2 7}) \approx O(n^{2.81})$
 - Space: More memory required for temporary submatrices

Observation Table

Matrix Size (n)	Conventional (ms)	Strassen (ms)
2x2	0.001	0.002
4x4	0.005	0.006
8x8	0.030	0.020
16x16	0.250	0.150

Comparison

- Strassen is asymptotically faster than conventional.
- Strassen requires more memory and is efficient for large matrices.
- Conventional method is simpler and preferred for small matrices.

Viva Questions

- 1 What is the time complexity of Strassen's algorithm?
- 2 Why does Strassen use 7 multiplications instead of 8?
- 3 For which matrix sizes is Strassen more efficient?
- 4 Which algorithm is preferred in practice for small inputs?

Conclusion

- Strassen reduces the number of multiplications at the cost of extra additions.
- Asymptotically faster than conventional method.
- For small matrices, conventional is faster due to lower overhead.

References

- Cormen et al., *Introduction to Algorithms*, MIT Press.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.